
Source Auditing Lecture Material

Course: *Introduction to Vulnerability Research*

Author name: *Dr. Bernard Lampe*

Due date: *October 17th, 2019*

Introduction

1. The primary skill of source code auditing is identifying which code may be vulnerable and is reachable from an exposed attack surface. Experienced code auditors are continually asking themselves the following questions:
 - (a) "Where should the limited time auditing be spent to maximize the chances of bug discovery?"
 - (b) "What pieces of code should be checked thoroughly and which can be ignored?"
 - (c) "Upon finding a bug, should more time be spent exploring to find more bugs, or should the focus turn to exploitation of the current set of bugs? (Exploration versus Exploitation Tradeoff)."
2. Large code bases make these questions difficult to answer. However, an experienced auditor will perform a thorough attack surface analysis and follow the logical flow of attacker data understanding and verifying each use of the data. General guidance to follow when starting any security audit is:
 - (a) Learn to read and write target language (C, C++, PHP, Java, etc).
 - (b) Know common bug patterns in the target language.
 - (c) Research existing CVE's and public research about the target.
 - (d) Find code which is in dis-use or rarely executed.
 - (e) Code is built in layers or modules. Scope the problem by looking at layers or modules individually and studying the API's.
 - (f) Find the ages of the code layers, bugs tend to be in the most recent or very old code.
 - (g) Find large back-end components of the software such as memory allocators, parsing functions/libraries, JIT engines, garbage collectors, etc.
 - (h) Understand the purpose of the code, what the author was trying to do and how the implementation falls short.
 - (i) Find the most complicated pieces of code with high cyclomatic complexity. Data structures which are used frequently or are polymorphically used (cast/unions) are important to understand.
 - (j) Get a feel for the code style and find places the style is violated. This can tell you if multiple authors are writing and merging code and opens up chances for mis-communications.

- (k) Need to know how high level code is translated into assembly looking for when the intended logical flow does not match the implementation.
- (l) Track down documentation
 - i. RFC's
 - ii. User manuals
 - iii. Client software
 - iv. Source leaks (even out of date code is valuable)
 - v. Bug trackers
 - vi. Code repositories/logs
 - vii. Stalk/blame bad developers
 - viii. Read all comments in source
 - ix. Find prior CVE's in public databases (exploitdb, cvedetails, etc)

Comparison of VR Techniques

1. Searching for vulnerabilities can be done using static or dynamic analysis. Static analysis includes source code auditing and binary reverse engineering. Dynamic analysis includes fuzz testing and debugging among other more advanced techniques.
2. Dynamic analysis
 - (a) Very few false positives (flagging bugs that don't exist)
 - (b) False negative rate (missed bugs) can be reduced by adding code coverage, feedback fuzzing or memory sanitizers
 - (c) Scaling can be done by adding more processing time
 - (d) Progress can be made using blackbox methods
 - (e) Able to find non-obvious bugs which utilize code from disparate parts of the source code base. This can make root causing hard.
 - (f) Special instrumentation is needed to find bugs that do not result in a crash.
3. Static analysis:
 - (a) Many more false positives (flagging bugs that don't exist) especially with automated tools
 - (b) False negative (missed bugs) rate is a function of the auditors understanding of the attack surface and code base
 - (c) Requires considerable computational/human resources to scale
 - (d) Progress requires considerable internal target knowledge
 - (e) Bugs found here typically include code which is localized about one area of the target.
 - (f) Source auditing can find non-memory corruption bugs (non-crashing) which are missed by fuzzing techniques.

Attack Surface Analysis

1. Attack surface analysis is the process of identifying the points of entry for any untrusted data. Knowing where incoming data is processed will dictate how an attack scenario proceeds. Attack surface analysis is valuable to source auditing to answer, "Where do I start auditing?", "What code is accessible and what code can be ignored?", "Has this code been audited or fuzzed before?", and "How many vulnerabilities have been found in this component before? Are there any left?".
2. Program input vectors are:
 - (a) network sockets (BSD-style sockets)
 - (b) inter-process communication (UNIX domain sockets, binder, signals, shared memory)
 - (c) memory mapped pages from the operating system (think dirty-cow)
 - (d) files on the filesystem
 - (e) environment variables
 - (f) software or firmware update channels
3. An important thing to keep in mind when auditing is what options were used at compile time. This will help you ignore code not compiled into the target.
4. Network servers or operating systems are highly configurable and some code may be present in the source base, but not compiled into the running target binary
5. Software such as web servers are highly extensible meaning other software modules may or may not be present. These modules are sometimes more vulnerable because they are written by third parties.
6. Some software has features available in debug builds which are not present in the release and the opposite is true as well. For example, assert statements in Javascript engines are compiled out in release builds.

Bug Patterns Through Small Examples

1. Naive approach to source auditing is to find all the `memcpy`'s, `strcpy`'s or other patterns and then trace the parameters back to their origin. This can be effective for smaller targets, but is not sufficient and untenable for larger targets.
2. It is better to start from the input vector and read through the code which handles the input data. Trace the data and examine the data validation code looking for unsanitized input by ensuring all data is constrained properly.

Stack Overflow

1. The classic stack overflow is simple but illustrates the fundamental reason overflows are exploitable. Stacks store in-line metadata which is critical to the control flow of a program (i.e. function return addresses and stack frame pointers).

2. Stacks have been exhaustively studied and modern stack overflows are hard to exploit due to fortify source protections inserted by the compiler. Stack cookies, stack variable re-ordering, safe string handling functions (think `strncpy`, `snprintf`, etc) and moving libraries to non-ascii address ranges are all mitigations to prevent exploitation.
3. Exploitation can still occur if interesting local variables exist on the stack and are referenced before the stack cookie is checked.
4. Noticing statically sized stack buffers is useful in finding such bugs. In addition, mis-use of the stack allocation routine `alloca()` can lead to a stack overflow.
5. A stack overflow is different from stack exhaustion. Stack exhaustion occurs when the stack runs out of space due to large allocations or deep function recursions.
6. One interesting strategy that continues to be exploitable is if you can write bytes to the stack at predictable offsets. If so, then overwriting pointers on the stack is useful to shift stack frames pointers or pointers to objects on the stack up or down into perhaps attacker controlled areas.
7. The following example of a stack corruption bug is based on CVE-2018-7445. Note that the `alloca()` (in Linux) will allocate a buffer on the stack by subtracting the size passed in from the stack pointer. This bug pattern is not possible in Windows because the `alloca` implementation will allocate a single page at a time and dereference each page incrementally.

Listing 1: Stack Corruption

```
1 void read_contents(int fd, unsigned int content_length_value)
2 {
3     unsigned char *stack_buffer;
4
5     // consider what happens when content_length_value = -1
6     stack_buffer = alloca(content_length_value);
7     read(fd, stack_buffer, content_length_value);
8 }
```

Heap Overflow

1. Exploiting a heap overflow requires knowing the implementation details. The implementation details will dictate the exploitation approach and what exploitation mitigations are present.
2. All heap implementations follow the POSIX API or `malloc()`, `calloc()`, `realloc()`, `free()`, etc. There are many heap allocators:
 - (a) `dlmalloc` (General)
 - (b) `ptmalloc2` (Glibc)
 - (c) `jemalloc` (FreeBSD, Firefox)

- (d) tcmalloc (Google)
 - (e) libumem (Solaris)
3. Ideal heap vulnerabilities consist of several qualities of which a subset may be present.
- (a) Control of the heap allocation size
 - (b) Control of the data overflowing the buffer
 - (c) Control of the length overflowed out-of-bounds
 - (d) Control of the life-time of the allocation (able to invoke when allocation and free occur)
 - (e) Ability to allocate and groom the heap with interesting objects containing meta-data (i.e. function pointers)
 - (f) Ability to invoke the use of useful meta-data that is overflowed.
4. The following example illustrates a function with an obvious heap overflow in the CREATE command. This simple example illustrates control of allocation size, length of overflow, life-time. This bug would require another heap primitive to overflow into containing metadata. In addition, the overflowed data is ASCII limiting null bytes.

Listing 2: Heap Corruption

```
1 #define MAX_STRS 0x100
2 #define STRLEN_MAX 0x100
3 enum cmd { CREATE = 0x10, DELETE, FIND };
4 char *g_strs[MAX_STRS];
5 int n_strs= 0;
6
7 char *find(char *str)
8 {
9     for(int i = 0; i < n_strs; i++) {
10         if (!strcmp(str, g_strs[i])) return g_strs[i];
11     }
12     return NULL;
13 }
14
15 // called repeatedly with user controlled command and c-string
16 char* str_store(enum cmd c, char *str)
17 {
18     char *ret = NULL;
19     switch (c) {
20         case CREATE:
21             if (n_strs < MAX_STRS) {
22                 g_strs[n_strs] = malloc(STRLEN_MAX);
23                 strcpy(g_strs[n_strs], str);
24                 ret = g_strs[n_strs++];
25             }
26             break;
27         case DELETE:
28             free(find(str));
29             break;
30         case FIND:
31             ret = find(str);
32             break;
33     }
34
35     return ret;
36 }
```

Format String Bugs

1. Format strings are not found often, because they are easy for compilers to programmatically check for. The compiler can easily check if the format string is not a constant string and can also check if the number of parameters in the format string matches the number and type of arguments. This prevents the class of bug entirely.
2. Also, FORTIFY_SOURCE features of gcc make these bugs hard to exploit. However, there are automated libraries for exploiting these bugs now.
3. If the attacker has control of the format string passed to printf, fprintf, vsprintf,

sprintf, etc, then he can read memory OOB, dereference memory OOB, and with %n feature can write attacker controlled data OOB.

4. The %n is widely disabled these days.

Listing 3: Format Strings

```
1 // based on CVE-2012-0809
2 void format_string(char *user_controlled)
3 {
4     int foo = 1;
5
6     // very bad, what would "%s", "%n" or "%x" do?
7     printf(user_controlled);
8
9     // what does the "%s" do?
10    printf("User_string_is_%s");
11
12    // what does the second "%x" print?
13    printf("User_variable_is_var=%x%x", foo);
14 }
```

Out-of-bounds (OOB) Read/Write

1. These can be caused by unchecked offsets, and can be relative or absolute.
2. A relative read/write is when the offset is added to a pointer dynamically allocated.
3. If the offsets are signed, then you can read/write relative before the buffer. If not, you can only write after. This will affect your memory grooming technique used.
4. An absolute read/write is when the attacker controls the dereferenced pointer being written to (i.e. can write to any address specified directly by the attacker).

Listing 4: Out-of-bound Read

```
1 #define MAX_SIZE 0x1000
2 char *g_buf;
3 void oob(uint8_t *buffer, uint32_t len, int32_t seek)
4 {
5     if (len > MAX_SIZE) return;
6
7     if (!g_buf) {
8         g_buf = malloc(len);
9         if (!g_buf) return;
10    }
11
12    // what happens if user controls seek?
13    memcpy(g_buf, buffer + seek, len - seek);
14 }
```

Arbitrary free

1. If the attacker can control the pointer value passed to free.
2. This can often times be turned into a use-after-free (UAF) by free'ing memory still in use.

Listing 5: Arbitrary Free

```
1 typedef struct Obj {
2     char buf[0x100];
3     struct Obj *next;
4 } Obj;
5 Obj list_head; // sentinel
6
7 void add_object(char *name) {
8     // set list name, what does this overwrite?
9     Obj *o = malloc(sizeof(Obj));
10    o->next = NULL;
11    strcpy(o->buf, name);
12
13    Obj *c = &list_head;
14    while(c->next)
15        c = c->next;
16    c->next = o;
17 }
18
19 void free_list() {
20     Obj *c = list_head.next;
21     do {
22         Obj *t = c;
23         c = c->next; // advance pointer
24         free(t); // free current
25     } while(c);
26 }
```

Double Free

1. A double free will add the same pointer to the heap allocator's free list twice.
2. Then two new allocations will be served the same free memory block which could result in a type confusion.
3. Most libc implementation will catch this because it's a simple check.

Listing 6: Double Free

```
1 typedef struct Obj {
2     char buf[0x100];
3     struct Obj *next;
4 } Obj;
5 Obj list_head; // sentinel
6
7 void add_object(char *name)
8 {
9     // setup object
10    Obj *o = calloc(1, sizeof(Obj));
11    strncpy(o->buf, name, 0xff);
12    o->next = NULL;
13
14    // add to list
15    Obj *c = &list_head;
16    while(c->next)
17        c = c->next;
18    c->next = o;
19 }
20
21 void remove_object(char *name)
22 {
23     Obj *c = list_head.next;
24     while(strncmp(c->buf, name, 0xff)) {
25         c = c->next;
26     }
27
28     // what does this case forget to do?
29     if (c) {
30         free(c);
31     }
32 }
33
34 void free_list()
35 {
36     Obj *c = list_head.next;
37     do {
38         Obj *t = c;
39         c = c->next; // advance pointer
40         free(t); // free current
41     } while(c);
42 }
43
44 int main()
45 {
46     // what is the state of memory after these calls?
47     add_object("foo");
48     remove_object("foo");
49
50     // what happens when we free the list?
51     free_list();
52 }
```

Use-after-free (UAF)

1. A use-after-free (UAF) is problematic because an attacker may be able to replace the memory.
2. Critical aspects of exploitability of these bugs are the ability to spray or groom the heap to replace the memory with attacker controlled data. Then having that attacker controlled way used in an interesting way.

Listing 7: Use-after-free

```
1 typedef struct Obj {
2     char buf[0x100];
3     struct Obj *next;
4 } Obj;
5 Obj list_head; // sentinel
6
7 void add_object(char *name) {
8     Obj *o = calloc(1, sizeof(Obj));
9     strncpy(o->buf, name, 0xff);
10    o->next = NULL;
11
12    Obj *c = &list_head;
13    while(c->next) c = c->next;
14    c->next = o;
15 }
16
17 void remove_object(char *name) {
18     Obj *c = list_head.next;
19     while(strncmp(c->buf, name, 0xff)) c = c->next;
20
21     // what does this case forget to do?
22     if (c) free(c);
23 }
24
25 void print_list() {
26     Obj *c = list_head;
27     while(c->next) {
28         printf("%s\n", c->buf);
29         c = c->next;
30     }
31 }
32
33 int main() {
34     // what is the state of memory after these calls?
35     add_object("foo");
36     add_object("bar");
37     remove_object("foo");
38
39     // what happens when we do this?
40     print_list();
41 }
```

Integer Overflow Example

1. The following code snippet is inspired by CVE-2015-3864 which is the Android StageFright bug in the MP4 parser.
2. The inputs to the function below are attacker controlled. Therefore, the addition and multiplication can overflow the size of a 32 bit unsigned integer. This results in the allocation of fewer bytes than necessary.
3. The memcpy will then copy data into the heap. Notice that the in_buf_size parameter is large to hit the overflow meaning the memcpy will copy large amounts of data and most likely segfault the process when attempting to write to unallocated/unmapped pages.
4. Many times one bug type can be transformed into another bug type. Here an integer overflow enabled a heap overflow.

Listing 8: Integer Overflow Code

```
1 void copy_buffer(unsigned char *in_buf, unsigned len)
2 {
3     unsigned char *buffer;
4     unsigned int buf_size;
5
6     buf_size = len + 16; // integer overflow
7     buffer = (unsigned char*)malloc(buf_size *
8                                     sizeof(unsigned char));
9     if (buffer != NULL) {
10         memcpy(buffer, in_buf, len);
11     }
12 }
```

Signedness Issues

1. Signedness issues can occur when an attacker can influence an integer type to be negative.
2. It is important that the developers are aware of integer casting rules to prevent these issues.

Listing 9: Signedness Issues

```
1 void signed(unsigned char *in_buf, int len)
2 {
3     if (len < 0) return;
4
5     // what happens if len == 0?
6     for(int i = len-1; i != 0; --i) {
7         in_buf[i] = to_upper(in_buf[i]);
8     }
9 }
```

Type Confusions

1. Type confusions are more often found in languages where polymorphism is built in such as C++.
2. However, type confusion can occur where type casting occurs. Look out for explicit cast operators in C and reinterpret_type() calls in C++. These circumvent type safety semantic checking in the compilers.

Listing 10: Type Confusion

```
1 #define NAME_TYPE 1
2 #define ID_TYPE 2
3
4 struct MessageBuffer
5 {
6     int msgType;
7     union {
8         char *name;
9         int nameID;
10    };
11 };
12
13 int main (int argc, char **argv) {
14     struct MessageBuffer buf;
15     char *defaultMessage = "Hello World";
16
17     buf.msgType = NAME_TYPE;
18     buf.name = defaultMessage;
19
20     if (argc > 1) {
21         buf.msgType = atoi(argv[1]);
22         buf.name = argv[2];
23     }
24
25     // what happens when the wrong type is given?
26     if (buf.msgType == NAME_TYPE) {
27         printf("Message: %s\n", buf.name);
28     }
29     else {
30         printf("Message: Use ID %x\n", buf.nameID);
31     }
32
33     return 0;
34 }
```

Information Leaks/Disclosure

1. The most infamous information disclosure is HeartBleed (CVE-2014-0346) in OpenSSL which would allow a remote attacker to dump heap data from a server process.
2. It was called HeartBleed because the heart beat (hello I'm alive) packet in the TLS was poorly implemented in the library.

3. This is an example of when a bug isn't found because the attack surface goes unnoticed. Typically new attack surfaces (no matter how simple) can harbor bugs.

Listing 11: Info Leak

```
1 // based on CVE-2014-0346, the attacker controls payload
2 // so it can be increased beyond the size of pl and returned
3 // to the attacker over the socket.
4 typedef struct payload {
5     char pl[0x10];
6 };
7 struct payload g_pl;
8 void send_reply(int fd, int payload)
9 {
10     char *bp = malloc(payload);
11     memcpy(bp, g_pl.pl, payload);
12     write(fd, bp, payload);
13 }
```

Command Injection

1. Command injection comes from using unsanitized input. When calling out to system via system(), popen(), execve(), etc, any shell escape characters need to be filtered out.

Listing 12: Command Injection

```
1 void unlink_file(char *fname)
2 {
3     char buf[1024];
4     struct stat statbuf;
5
6     // get file stats
7     if (!stat(fname, &statbuf)) return;
8
9     // check if regular file
10    if (statbuf.st_mode & S_IFMT == S_IFREG) {
11        // what happens if fname has shell escape characters is in it?
12        snprintf(buf, 1024, "/bin/rm_%s", fname);
13
14        FILE *fp = popen(buf, "r");
15        if (fp == NULL) return;
16        pclose(fp);
17    }
18 }
```

Race Condition

1. Race conditions, also calls TOUTOC (for Time-of-Use,Time-of-Check), are bugs in concurrent programs (multi-threading or multi-process) where shared state is not property synchronized with the use of locking mechanisms.

2. These are hard to check for using static analysis techniques such as source code auditing.
3. When you see locks being used or state being shared, ask "What memory and variables are shared?" and "What synchronization mechanisms are used to serialize access (spin locks, event locks, reader/writer locks, semaphores)?"
4. Some race conditions are very hard to hit. Also look for global locks which will be in higher function calls. These are sometimes used as band-aids for the developers when run time is not an issue. Past Linux versions and Python2 used global locks at runtime.
5. Some race conditions are very hard to hit. Also look for global locks which will be in higher function calls. These are sometimes used as band-aids for the developers when run time is not an issue. Past Linux versions and Python2 used global locks at runtime.
6. Dirty Cow (CVE-2016-5195) is a very infamous race condition but too elaborate to go into here.

Listing 13: Race Condition

```
1 // what happens if we run this and have another process
2 // switch out the file after the check?
3 void racer(unsigned char *fname)
4 {
5     if (!access(fname, R_OK | W_OK)) {
6         perror("file_permission_denied");
7         return;
8     }
9
10    // read and write file
11 }
```

Logic Error

1. Logic error due to a bad code merge or copy and paste error.
2. Permits MitM attacker to capture and modify data in sessions protected by SSL/TLS by short circuiting the SHA1 validation.

Listing 14: Logic Error

```
1 void SSLVerifySignedServerKeyExchange()
2 {
3     // <snip'ed code>
4     ashOut.data = hashes + SSL_MD5_DIGEST_LEN;
5     hashOut.length = SSL_SHA1_DIGEST_LEN;
6     if ((err = SSLFreeBuffer(&hashCtx)) != 0)
7         goto fail;
8     if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
9         goto fail;
10    if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
11        goto fail;
12    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
13        goto fail;
14    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
15        goto fail;
16    goto fail; // what does this line do?
17    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
18        goto fail;
19
20    err = sslRawVerify(...);
21    // <snip'ed code>
22 }
```

REF Counting

1. Reference counting can be viewed as a memory allocator with amortized work. All references to a single object use the same underlying structure/object memory. Allocation happens during initialization (RAII) and free'ing happens when reference count goes to zero.
 - (a) A put operations is decrementing (free'ing) a ref count. When refcount goes to zero, then memory is free'd.
 - (b) A get operation is incrementing (allocating) a ref count.
2. Mis-counting a refcount can lead to UAF.
 - (a) If an attacker can invoke an extra put, then a UAF can be directly achieved.
 - (b) If an attacker can invoke an extra get, then the attacker can integer overflow the refcount to zero and achieve a UAF.

Listing 15: Incorrect Reference Counting

```
1 // reference counting structure
2 struct ref {
3     void (*free)(const struct ref *);
4     int count;
5 };
6
7 void ref_inc(const struct ref *ref)
8 {
9     ((struct ref *)ref)->count++;
10 }
11
12 void ref_dec(const struct ref *ref)
13 {
14     if (--((struct ref *)ref)->count == 0)
15         ref->free(ref);
16 }
17
18 // embed reference count in a struct/class
19 typedef struct Obj{
20     int data;
21     struct ref refcount;
22 } Obj;
23
24 // struct/class initialization
25 Obj *alloc_obj(int data) {
26     // allocate memory
27     Obj *o = calloc(1, sizeof(Obj));
28     o->data = data;
29
30     // init ref count
31     o->ref.free = free;
32     ref_inc(o->ref);
33
34     return o;
35 }
36
37 int main(int argc, char **argv)
38 {
39     if (argc != 2) return -1;
40     uint32_t num_to_alloc = strtoul(argv[1], NULL, 10);
41
42     // what happens when num_to_alloc = 0xFFFFFFFF?
43     Obj *o = alloc_obj(42);
44     for(int i = 0; i < num_to_alloc, i++) {
45         ref_inc(o->refcount);
46     }
47
48     // de-allocations
49     for(int i = 0; i < num_to_alloc, i++) {
50         ref_dec(o->refcount);
51     }
52
53     return 0;
54 }
55 }
```

Uninitialized Memory Bugs

1. Use of uninitialized memory can be problematic if the attacker can replace the memory on the stack or heap with their data via a groom or spray.

Listing 16: Memory Used Unintialized

```
1 #define ARRAY_SIZE 0x100
2 #define STR_SIZE 0x10
3 char **g_array;
4 void alloc_array(int len) {
5     if (!g_array) {
6         g_array = malloc(ARRAY_SIZE * sizeof(char*));
7         if (!g_array) return;
8     }
9
10    for(int i = 0; i < ARRAY_SIZE, i++) {
11        array[i] = calloc(STR_SIZE, sizeof(char*));
12    }
13 }
14
15 void use_array() {
16     if (!g_array) return;
17     for(int i = 0; i < ARRAY_SIZE; i++) {
18         printf("%s\n", g_array[i]);
19     }
20 }
21
22 void free_array() {
23     if (!g_array) return;
24     for(int i = 0; i < ARRAY_SIZE; i++) {
25         free(g_array[i]);
26     }
27     g_array[0] = NULL; // is this sufficient?
28 }
29
30 int main() {
31     alloc_array(10);
32     free_array(10);
33     use_array(10);
34 }
```

Integer Promotion Bugs

1. Another good exercise which will help you improve your source auditing code is knowing about integer promotion rules. I recommend using the quiz at => pleasestopnamingvulnerabilities.com/integers.html.
2. As a rule of thumb, the compiler emit assembly code so that the integers are promoted to the widest type and then compared.
3. Integer ranges:

- (a) signed char: -127 to 127
- (b) unsigned char: 0 to 255
- (c) signed short: -32767 to 32767
- (d) unsigned short: 0 to 65535
- (e) signed int: -2147483647 to 2147483647 */

Listing 17: Memory Used Uninitialized

```
1 //Why does this give a strange, large integer number and not 255?
2 unsigned char x = 0;
3 unsigned char y = 1;
4 printf("%u\n", x - y);
5
6 // Why does this give "-1 is larger than 0"?
7 unsigned int a = 1;
8 signed int b = -2;
9 if(a + b > 0)
10     puts("-1 is larger than 0");
11
12 // Why does changing the type in the above example to short fix the problem?
13 unsigned short a = 1;
14 signed short b = -2;
15 if(a + b > 0)
16     puts("-1 is larger than 0"); // will not print
```

Source Auditing Tools

1. Source auditing tools can improve productivity considerably by reducing time to perform attack surface analysis, find definitions, locate calling functions, and understanding of logical flow.
2. Critical features of any source auditing tool are as follows.
 - (a) Navigating cross references (ctags, csope)
 - i. Searching for definitions of functions, classes, structs, types, etc
 - ii. Searching for places a function is called or referenced
 - (b) Understanding preprocessor directives and expand complicated macros
 - (c) Search by file name (sublime text)
 - (d) Search by struct or object type (eclipse)
 - (e) Meld is a handy tool to diff source files
 - (f) Joern is newer tool that allows for writing complicated queries to find vulnerable patterns in abstract syntax trees
 - (g) Visual studios, Eclipse and Understand are all excellent source tools. Only Eclipse is free the others are paid.

Conclusion

Understanding code well enough to find bugs is a difficult problem which is why code reviews are still the most effective way for developers to avoid releasing buggy code. The most advanced automated source analysis tools such as those from Grammatech employ constraint solving and code simulation environments. However, all such tools suffer from high false-positive rates. Understanding which code is relevant in an attack scenario and what constitutes an exploitable bug is difficult to do in an automated fashion.

References

1. blog.nsogroup.com, A tale of two mallocs
2. cscope.sourceforge.net
3. ctags.sourceforge.net
4. exploit-exercises.lains.space
5. joern.io
6. meldmerge.org
7. pleasestopnamingvulnerabilities.com/integers.html
8. scitools.com (understand tool)
9. visualstudio.microsoft.com
10. www.eclipse.org
11. www.fuzzingbook.com
12. www.grammatech.com (codesonar tool)
13. A Bug Hunter's Diary, Book by Tobias Klein
14. The Art of Software Security Assessment, Book by John McDonald, Justin Schuh, and Mark Dowd